

TAREA 3.0

1. INVESTIGACIÓN

1. Control de Versiones

Definición

Es un sistema que permite registrar y controlar los cambios realizados sobre archivos o proyectos a lo largo del tiempo.

Utilidad

- Mantiene un historial de cambios.
- Permite recuperar versiones anteriores.
- Facilita el trabajo colaborativo.

Ejemplo Práctico

Dos desarrolladores trabajan sobre el mismo proyecto y pueden combinar sus cambios sin sobrescribir el trabajo del otro.

Fuente Bibliográfica

- Chacon, S., & Straub, B. (2014). Pro Git. Apress.

2. Git

Definición

Es un sistema de control de versiones distribuido creado por Linus Torvalds en 2005.

Utilidad

- Gestiona cambios en el código fuente.
- Permite trabajar sin conexión.
- Facilita el trabajo en equipo.

Ejemplo Práctico

```
git init
git add .
git commit -m "Primer commit"
```

Fuente Bibliográfica

- <https://git-scm.com/doc>

3. GitHub

Definición

Es una plataforma basada en la nube que permite almacenar repositorios Git y colaborar en proyectos.

Utilidad

- Hospedaje de código.
- Colaboración entre desarrolladores.
- Automatización mediante GitHub Actions.

Ejemplo Práctico

Crear un repositorio y compartirlo con un compañero para desarrollar una aplicación.

Fuente Bibliográfica

- <https://github.com>

4. Repositorio Local

Definición

Es la copia del proyecto almacenada en la computadora del desarrollador.

Utilidad

Permite realizar cambios y pruebas antes de publicarlos.

Ejemplo Práctico

```
git clone https://github.com/usuario/proyecto.git
```

Fuente Bibliográfica

- Pro Git, Capítulo 2.

5. Repositorio Remoto

Definición

Es la versión del proyecto almacenada en un servidor como GitHub.

Utilidad

Permite compartir y respaldar el código.

Ejemplo Práctico

```
git push origin main
```

Fuente Bibliográfica

- Git Documentation.

6. Commit

Definición

Es un registro de cambios realizados en el repositorio.

Utilidad

Permite identificar y rastrear modificaciones.

Ejemplo Práctico

```
git commit -m "Agrega formulario de login"
```

Fuente Bibliográfica

- Git Documentation.

7. Branch (Rama)

Definición

Es una línea independiente de desarrollo dentro del repositorio.

Utilidad

Permite trabajar en nuevas funcionalidades sin afectar la rama principal.

Ejemplo Práctico

```
git checkout -b feature-login
```

Fuente Bibliográfica

- Pro Git.

8. Merge

Definición

Proceso de combinar cambios de una rama en otra.

Utilidad

Integra nuevas funcionalidades al proyecto principal.

Ejemplo Práctico

```
git merge feature-login
```

Fuente Bibliográfica

- Git Documentation.

9. Pull Request

Definición

Solicitud para integrar cambios de una rama a otra.

Utilidad

Permite revisar y aprobar código antes de fusionarlo.

Ejemplo Práctico

Un desarrollador crea un Pull Request para integrar una nueva funcionalidad a la rama main.

Fuente Bibliográfica

- GitHub Docs.

10. Fork

Definición

Es una copia de un repositorio en otra cuenta de GitHub.

Utilidad

Permite colaborar en proyectos sin afectar el repositorio original.

Ejemplo Práctico

Realizar un fork de un proyecto Open Source para contribuir con mejoras.

Fuente Bibliográfica

- GitHub Docs.

11. Conflictos de Integración

Definición

Ocurren cuando Git no puede determinar automáticamente qué cambios conservar.

Utilidad

Ayudan a detectar modificaciones incompatibles.

Ejemplo Práctico

Dos desarrolladores modifican la misma línea de código.

Fuente Bibliográfica

- Git Documentation.

12. Integración Continua (CI)

Definición

Práctica que automatiza la compilación y pruebas del software.

Utilidad

Detecta errores tempranamente.

Ejemplo Práctico

GitHub Actions ejecuta pruebas automáticamente al realizar un Push.

Fuente Bibliográfica

- Martin Fowler.

13. Entrega Continua (Continuous Delivery)

Definición

Práctica que mantiene el software listo para ser desplegado en cualquier momento.

Utilidad

Reduce riesgos en producción.

Ejemplo Práctico

Generar automáticamente paquetes listos para producción.

Fuente Bibliográfica

- Continuous Delivery (Humble & Farley).

14. Despliegue Continuo (Continuous Deployment)

Definición

Extensión de Continuous Delivery donde los cambios aprobados se despliegan automáticamente.

Utilidad

Acelera la entrega de software.

Ejemplo Práctico

Cada cambio aprobado se publica automáticamente en un servidor.

Fuente Bibliográfica

- Continuous Delivery.

15. GitHub Actions

Definición

Servicio de automatización integrado en GitHub para CI/CD.

Utilidad

Permite ejecutar pruebas, compilaciones y despliegues automáticos.

Ejemplo Práctico

Archivo `.github/workflows/main.yml`

```
name: CI
```

```
on:
```

```

push:
  branches:
    - main

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Verificar repositorio
        run: echo "Workflow ejecutado correctamente"

```

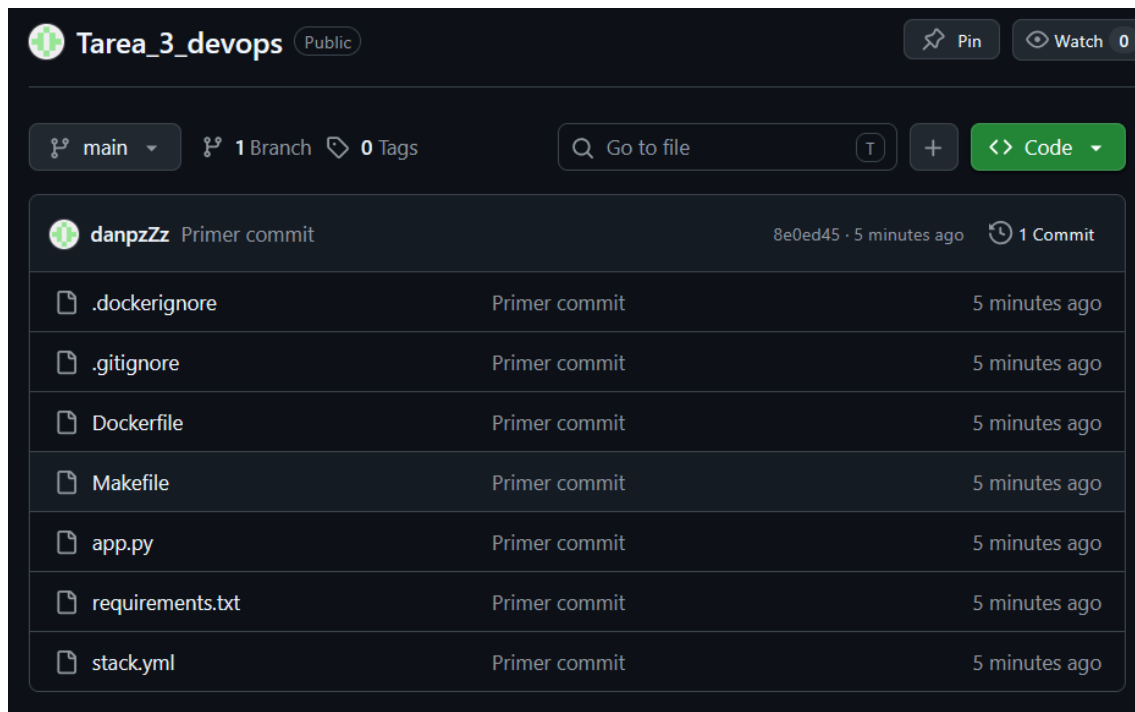
Fuente Bibliográfica

- <https://docs.github.com/actions>

2. Demostración Práctica

1. Creación del repositorio

Se creó el repositorio denominado "Tarea_3_devops" en GitHub, el cual será utilizado para almacenar y gestionar el proyecto desarrollado con Flask.



2. Estructura inicial del proyecto en Visual Studio Code

Se desarrolló la estructura base del proyecto utilizando Flask, incluyendo los archivos de configuración, dependencias y código fuente necesarios para la aplicación.

```
Compressing objects: 100% (5/5), done.
Writing objects: 100% (7/7), 2.26 KiB | 257.00 KiB/s, done.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/danpzZz/Tarea_3_devops.git
* [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

C:\Users\danpzZz\Desktop\tarea_3>git checkout -b estudiante-a
Switched to a new branch 'estudiante-a'

C:\Users\danpzZz\Desktop\tarea_3>git branch
* estudiante-a
main
```

3. Creación de la rama estudiante-a

Se creó la rama de trabajo denominada "estudiante-a" para implementar nuevas funcionalidades sin afectar la rama principal del proyecto.

Se registraron los cambios realizados en la rama "estudiante-a" mediante un commit descriptivo, permitiendo mantener el historial de modificaciones del proyecto.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

main

C:\Users\danpzZz\Desktop\tarea_3>git add .
warning: in the working copy of 'app.py', LF will be replaced by CRLF the next time Git touches it

C:\Users\danpzZz\Desktop\tarea_3>git commit -m "Agrega nueva pagina"
[estudiante-a c534c81] Agrega nueva pagina
1 file changed, 9 insertions(+), 1 deletion(-)

C:\Users\danpzZz\Desktop\tarea_3>git push origin estudiante-a
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 447 bytes | 447.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'estudiante-a' on GitHub by visiting:
remote:   https://github.com/danpzZz/Tarea_3_devops/pull/new/estudiante-a
remote:
To https://github.com/danpzZz/Tarea_3_devops.git
* [new branch]      estudiante-a -> estudiante-a

C:\Users\danpzZz\Desktop\tarea_3>
```

4. Creación de la rama estudiante-b

Descripción:

Se creó la rama "estudiante-b" para desarrollar nuevas funcionalidades de manera independiente y simular un entorno de trabajo colaborativo.

Se realizaron cambios en la aplicación y posteriormente se registraron mediante un commit para mantener el control de versiones.

```
C:\Users\danpzZz\Desktop\tarea_3>git checkout -b estudiante-b
Switched to a new branch 'estudiante-b'

C:\Users\danpzZz\Desktop\tarea_3>git add .

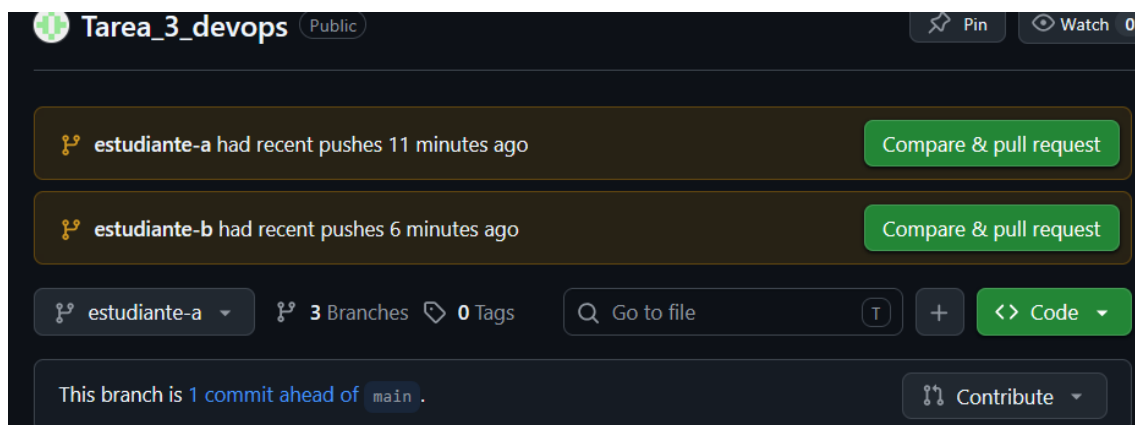
C:\Users\danpzZz\Desktop\tarea_3>git commit -m "Agrega pagina contacto"
[estudiante-b 96bb0b1] Agrega pagina contacto
1 file changed, 8 insertions(+), 1 deletion(-)

C:\Users\danpzZz\Desktop\tarea_3>git push origin estudiante-b
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 390 bytes | 390.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'estudiante-b' on GitHub by visiting:
remote:   https://github.com/danpzZz/Tarea_3_devops/pull/new/estudiante-b
remote:
To https://github.com/danpzZz/Tarea_3_devops.git
 * [new branch]      estudiante-b -> estudiante-b

C:\Users\danpzZz\Desktop\tarea_3>
```

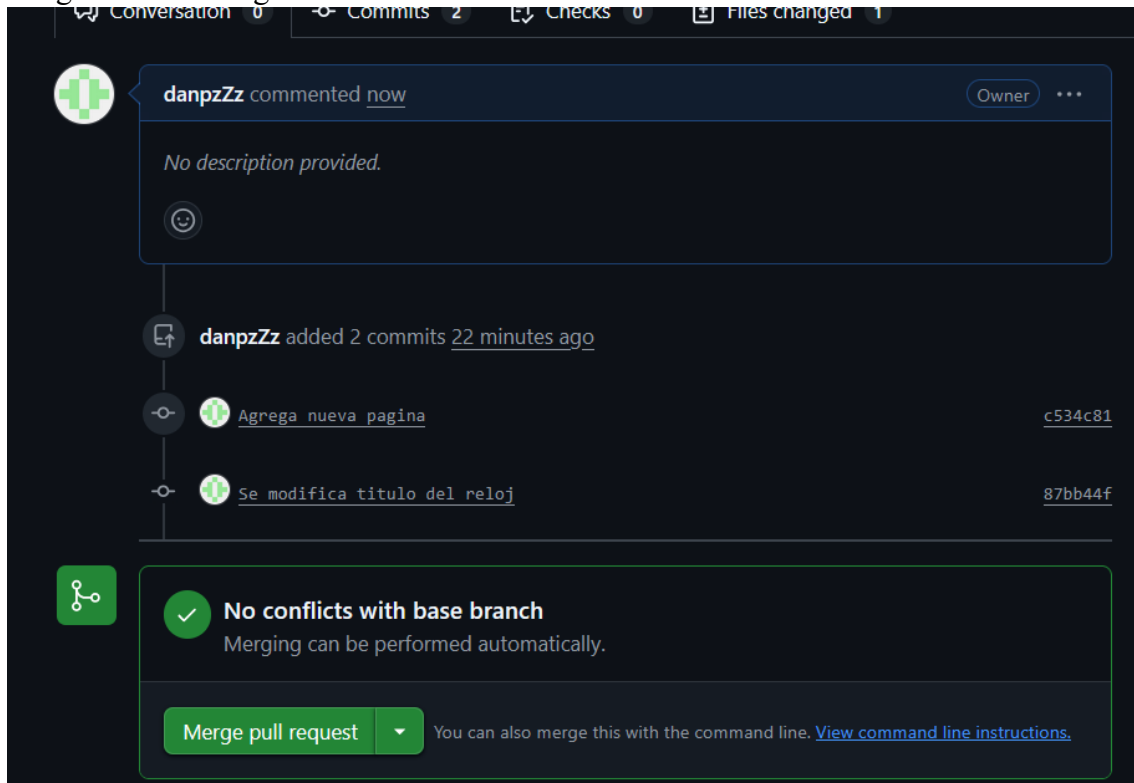
5. Publicación de Ramas en el Repositorio Remoto

Se publicaron las ramas **estudiante-a** y **estudiante-b** en GitHub utilizando el comando git push, permitiendo visualizar y gestionar los cambios desde el repositorio remoto.



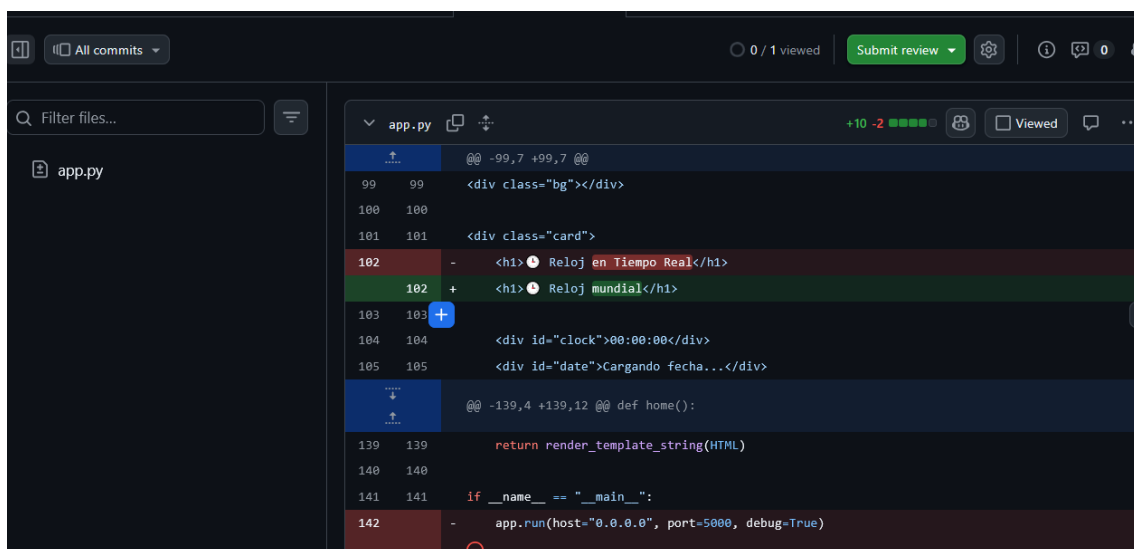
6. Pull Request de estudiante-a

Se creó un Pull Request para solicitar la integración de los cambios desarrollados en la rama **estudiante-a** hacia la rama principal **main**, iniciando el proceso de revisión e integración de código.



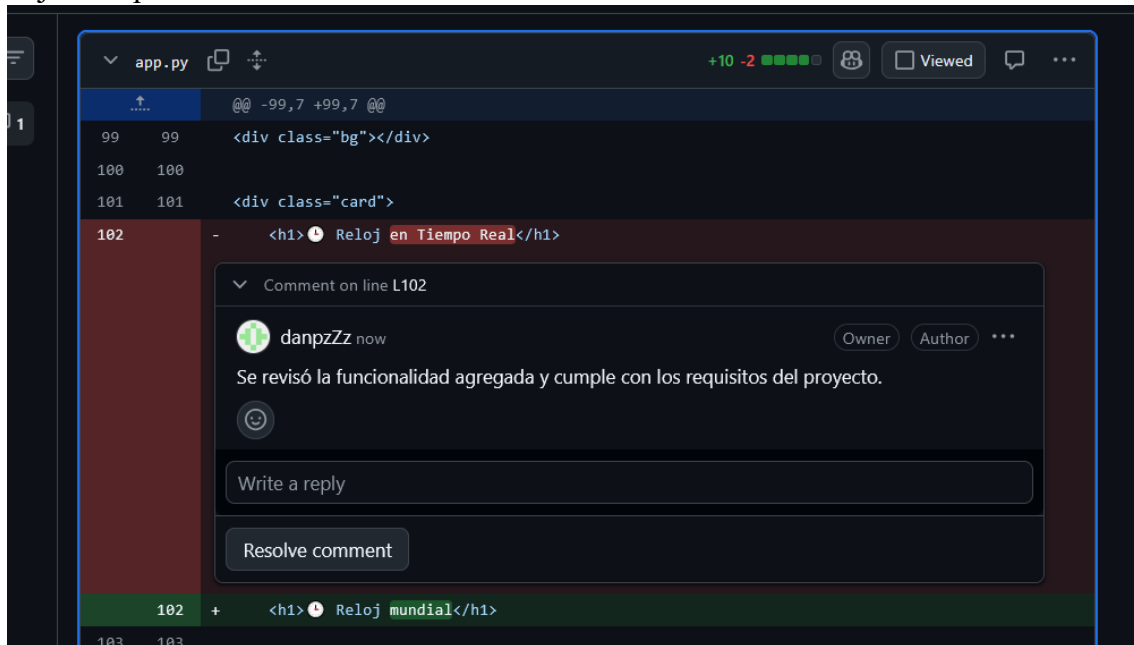
7. Revisión de Cambios en el Código Fuente

Se revisaron los cambios realizados en el archivo `app.py`, verificando que la funcionalidad implementada cumpla con los requisitos establecidos antes de proceder con su integración.



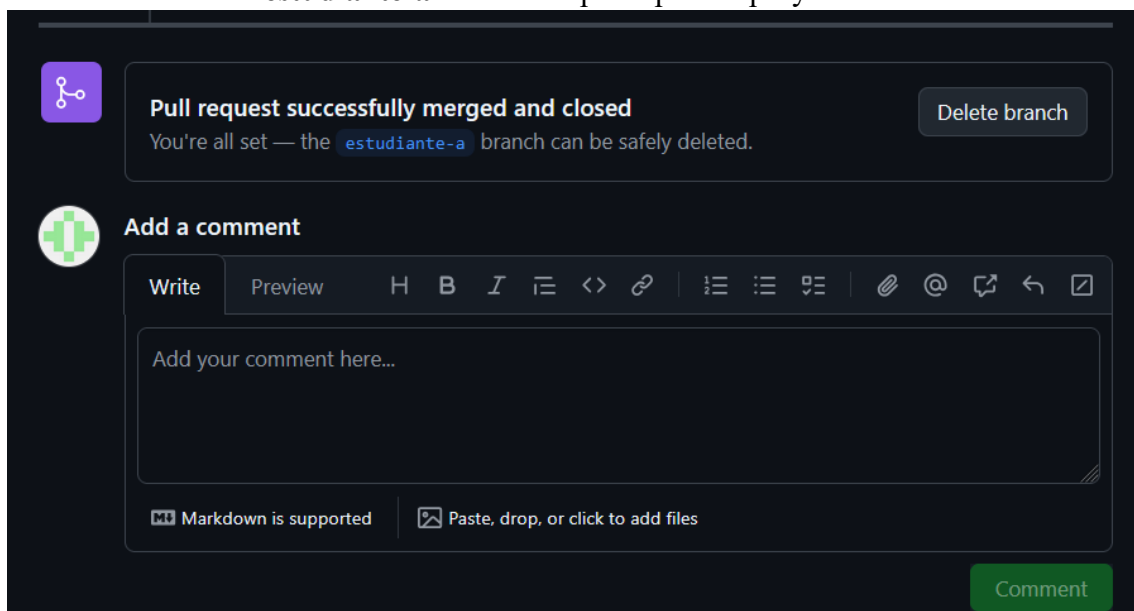
8. Proceso de Revisión y Validación de Código

Se realizó una revisión del código agregando comentarios dentro del Pull Request, validando que los cambios implementados funcionan correctamente y cumplen con los objetivos planteados.



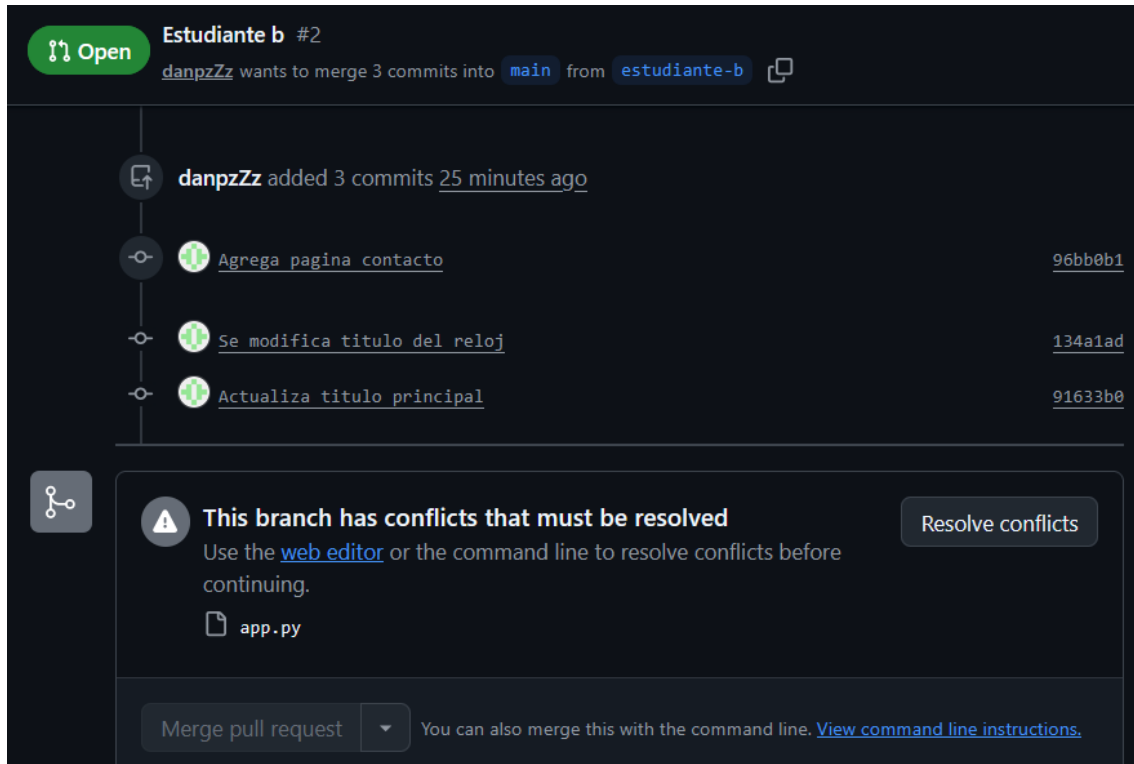
9. Integración de Cambios mediante Merge

Después de la revisión correspondiente, se realizó el proceso de **Merge**, integrando los cambios de la rama **estudiante-a** en la rama principal del proyecto.



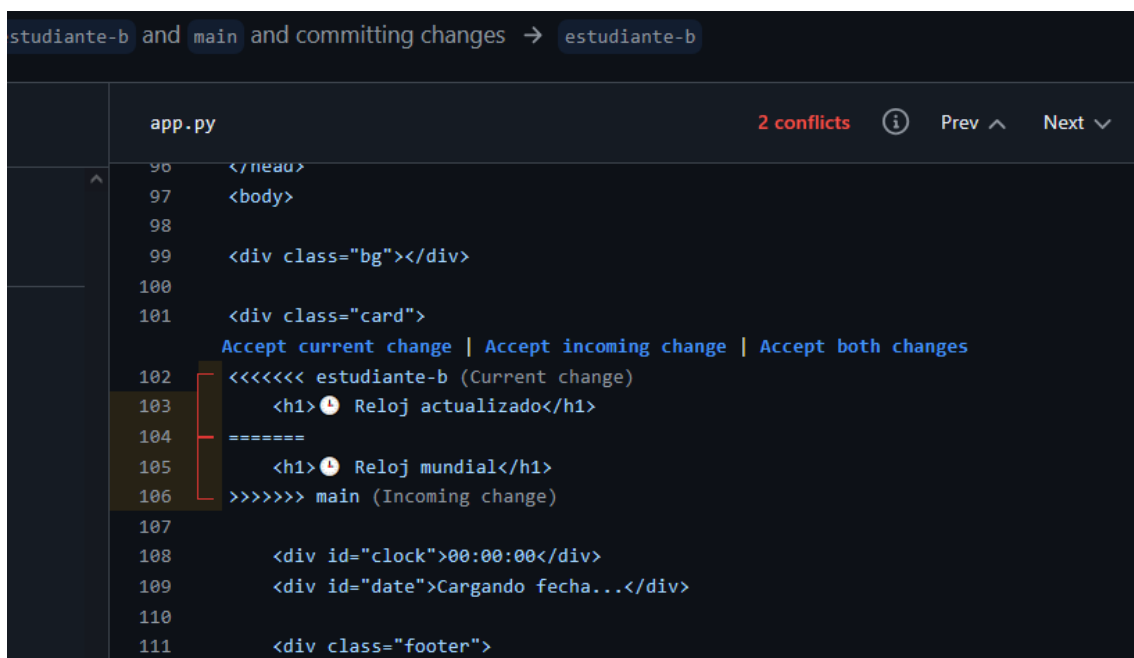
10. Detección de Conflicto de Integración

Durante el proceso de integración de la rama estudiante-b, GitHub detectó un conflicto debido a modificaciones realizadas sobre el mismo archivo, impidiendo el Merge automático.



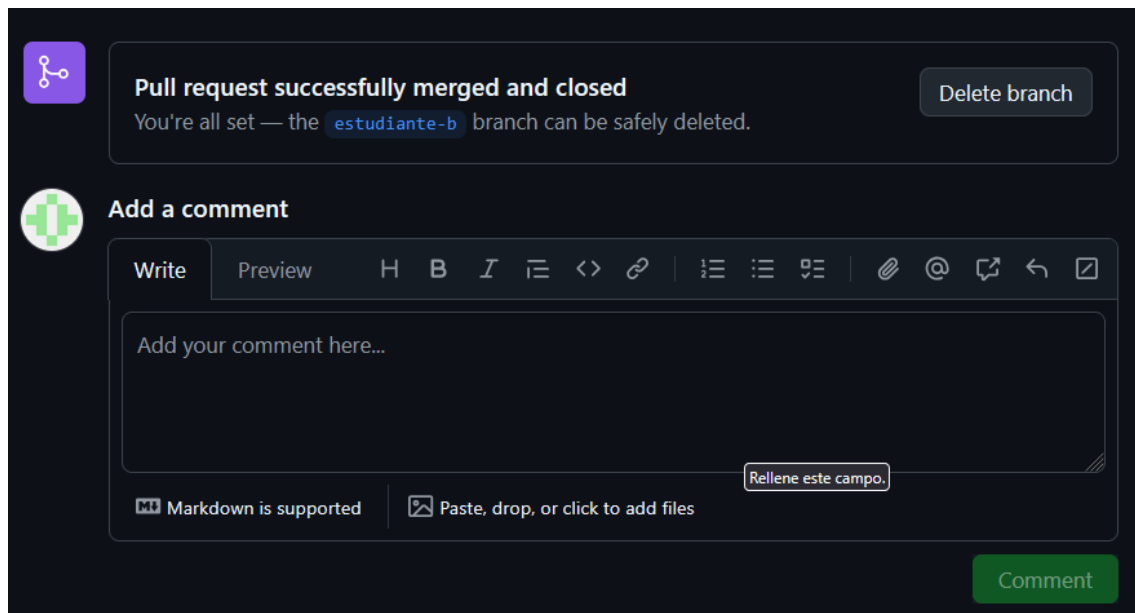
11. Resolución del Conflicto de Integración

Se analizaron los cambios en conflicto y se realizó una integración manual de las modificaciones, asegurando que ambas funcionalidades fueran conservadas correctamente dentro del proyecto.



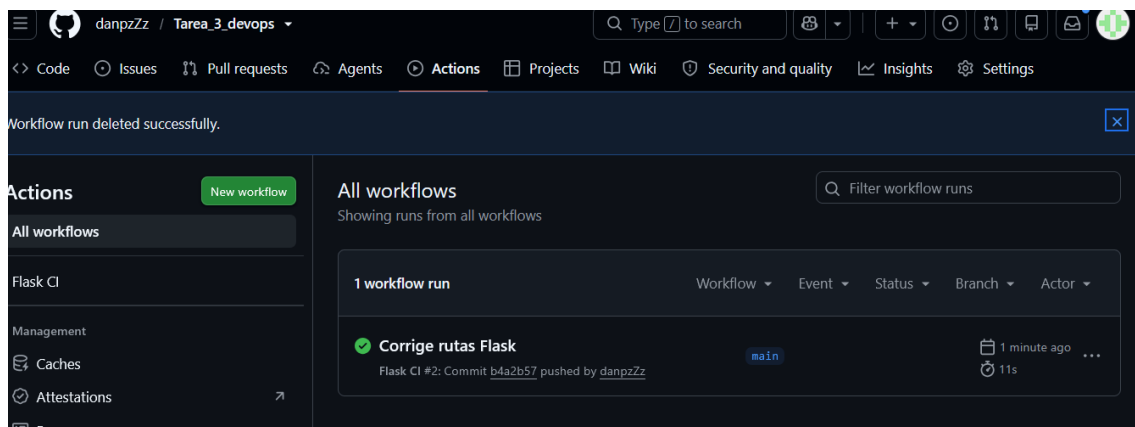
12. Merge Exitoso de la Rama "estudiante-b"

Una vez resuelto el conflicto de integración, se completó exitosamente el proceso de Merge, incorporando los cambios de la rama estudiante-b a la rama principal.



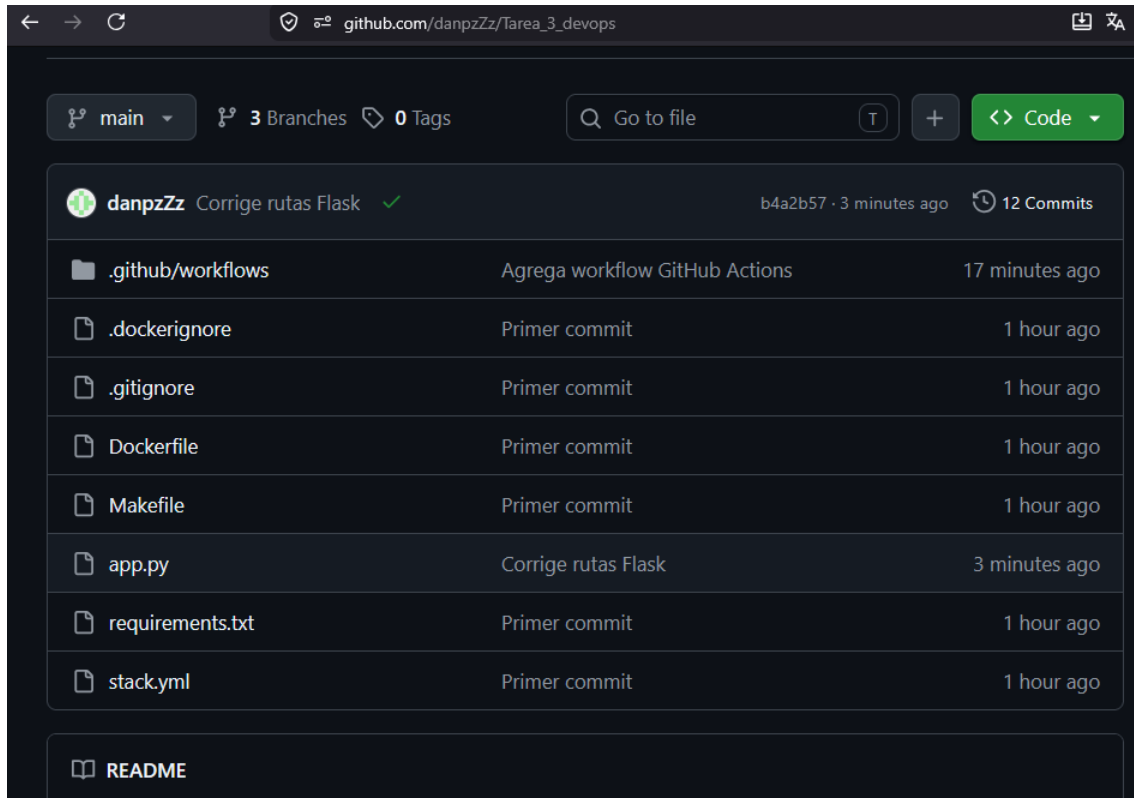
13. Implementación de GitHub Actions

Se configuró un workflow de GitHub Actions para automatizar la validación del proyecto, permitiendo verificar la sintaxis y el correcto funcionamiento de la aplicación cada vez que se realizan cambios en la rama principal.



14. Estado Final del Repositorio

Se muestra el estado final del repositorio una vez integradas todas las funcionalidades, resueltos los conflictos y ejecutadas exitosamente las validaciones automáticas mediante GitHub Actions.



Nombres completos de los integrantes.

Daniel Pérez

Carrera y paralelo.

Desarrollo de Software – Quinto C

URL del repositorio GitHub.

https://github.com/danpzZz/Tarea_3_devops.git

Conclusiones individuales de cada integrante (mínimo 5 líneas por estudiante).

Durante el desarrollo de esta práctica adquirí conocimientos fundamentales sobre el uso de Git y GitHub como herramientas de control de versiones para la gestión de proyectos de software. Aprendí a crear y administrar ramas de trabajo, realizar commits descriptivos, publicar cambios en repositorios remotos y utilizar Pull Requests para integrar nuevas funcionalidades. Además, comprendí la importancia de la revisión de código y la resolución de conflictos de integración para mantener la estabilidad del proyecto. La implementación de GitHub Actions me permitió conocer el concepto de integración continua y la automatización de validaciones dentro del ciclo de desarrollo.

En general, esta actividad fortaleció mis habilidades técnicas y me permitió comprender de manera práctica el flujo de trabajo utilizado en entornos profesionales de desarrollo de software.

Bibliografía utilizada.

1. Chacon, S., & Straub, B. (2014). Pro Git (2.^a ed.). Apress. Recuperado de: <https://git-scm.com/book/es/v2>
2. Git. (s.f.). Git Documentation. Recuperado de: <https://git-scm.com/doc>
3. GitHub. (s.f.). GitHub Documentation. Recuperado de: <https://docs.github.com>
4. Humble, J., & Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley Professional.
5. Flask. (s.f.). Flask Documentation. Recuperado de: <https://flask.palletsprojects.com>